

BackendConfig#

[https://pytorch.org/docs/stable/generated/torch.quantization.backend_conf](https://pytorch.org/docs/stable/generated/torch.quantization.backend_config)

Description:

Config that defines the set of patterns that can be quantized on a given backend, and how reference quantized models can be produced from these patterns. A pattern in this context refers to a module, a functional, an operator, or a directed acyclic graph of the above. Each pattern supported on the target backend can be individually configured through `BackendPatternConfig` in terms of: The supported input/output activation, weight, and bias data types How observers and quant/dequant ops are inserted in order to construct the reference pattern, and (Optionally) Fusion, QAT, and reference module mappings.

Function Signature

```
class
torch.ao.quantization.backend_config.BackendConfig(name='')
[source]#

property configs:
list[torch.ao.quantization.backend_config.backend_config.BackendPatternConfig]#

classmethod from_dict(backend_config_dict)[source]#

Return type
set_backend_pattern_config(config)[source]#
```

Returns:

dict[str, Any]

Code Examples

```
import torch
from torch.ao.quantization.backend_config import (
    BackendConfig,
    BackendPatternConfig,
    DTypeConfig,
    ObservationType,
)
```

```

weighted_int8_dtype_config = DTypeConfig(
    input_dtype=torch.quint8,
    output_dtype=torch.quint8,
    weight_dtype=torch.qint8,
    bias_dtype=torch.float)

def fuse_conv2d_relu(is_qat, conv, relu):
    return torch.ao.nn.intrinsic.ConvReLU2d(conv, relu)

# For quantizing Linear
linear_config = BackendPatternConfig(torch.nn.Linear)
.set_observation_type(ObservationType.OUTPUT_USE_DIFFERENT_OBSERVER_AS_INPUT)
.add_dtype_config(weighted_int8_dtype_config)
.set_root_module(torch.nn.Linear)
.set_qat_module(torch.ao.nn.qat.Linear)
.set_reference_quantized_module(torch.ao.nn.quantized.reference.Linear)

# For fusing Conv2d + ReLU into ConvReLU2d
conv_relu_config = BackendPatternConfig((torch.nn.Conv2d,
torch.nn.ReLU))
.set_observation_type(ObservationType.OUTPUT_USE_DIFFERENT_OBSERVER_AS_INPUT)
.add_dtype_config(weighted_int8_dtype_config)
.set_fused_module(torch.ao.nn.intrinsic.ConvReLU2d)
.set_fuser_method(fuse_conv2d_relu)

# For quantizing ConvReLU2d
fused_conv_relu_config =
BackendPatternConfig(torch.ao.nn.intrinsic.ConvReLU2d)
.set_observation_type(ObservationType.OUTPUT_USE_DIFFERENT_OBSERVER_AS_INPUT)
.add_dtype_config(weighted_int8_dtype_config)
.set_root_module(torch.nn.Conv2d)
.set_qat_module(torch.ao.nn.intrinsic.qat.ConvReLU2d)
.set_reference_quantized_module(torch.ao.nn.quantized.reference.Conv2d)

```

```
backend_config = BackendConfig("my_backend")
    .set_backend_pattern_config(linear_config)
    .set_backend_pattern_config(conv_relu_config)
    .set_backend_pattern_config(fused_conv_relu_config)
```

Detailed Documentation

Documentation

Config that defines the set of patterns that can be quantized on a given backend, and how reference quantized models can be produced from these patterns.

A pattern in this context refers to a module, a functional, an operator, or a directed acyclic graph of the above. Each pattern supported on the target backend can be individually configured through BackendPatternConfig in terms of:

The supported input/output activation, weight, and bias data types

How observers and quant/dequant ops are inserted in order to construct the reference pattern, and

(Optionally) Fusion, QAT, and reference module mappings.

The format of the patterns is described in: [pytorch/pytorch](#)

Return a copy of the list of configs set in this BackendConfig.

Create a BackendConfig from a dictionary with the following items:

“name”: the name of the target backend

“configs”: a list of dictionaries that each represents a BackendPatternConfig

Set the config for an pattern that can be run on the target backend. This overrides any existing config for the given pattern.

Set the configs for patterns that can be run on the target backend. This overrides any existing config for a given pattern if it was previously registered already.

Set the name of the target backend.

Convert this BackendConfig to a dictionary with the items described in from_dict().